THE CURIOSITY CUP 2026
A Global SAS® Student Competition

Reading the Court: A Comparative Study of Machine Learning Models for Basketball Shot Prediction

The LogReg Lakers

ABSTRACT

This study builds multi-season, shot-level NBA datasets to predict field goal success from contextual, player, and schedule features, using both complete-case and imputed versions to assess the impact of missing-data strategies. We compare regularized logistic regression, decision trees, and subsampled SVMs within a unified SAS–Python pipeline and find similar, moderate performance across models, with shot distance as the dominant predictor and game context, player size/role, and travel variables providing smaller effects. Overall, the results highlight substantial inherent randomness in shot outcomes but also reveal stable, interpretable patterns that form a robust baseline for more advanced sports analytics work.

INTRODUCTION

The National Basketball Association (NBA) generates detailed data for every field goal attempt: location on the court, time in the game, score situation, and information about the players involved. This makes it an ideal setting for studying which factors drive shot success and how accurately we can predict whether a given shot will go in. In this project, we build a large-scale shot-level dataset and use it to predict the probability that a shot attempt is made. Our main goals are (1) to quantify and compare the predictive value of contextual, player, and schedule features, and (2) to contrast simple, interpretable models with more flexible non-linear approaches to see how much extra performance complexity buys us.

DATA

Our starting point is a shot-level dataset covering multiple NBA regular seasons, where each row in the final modeling table corresponds to a single shot attempt. For each shot, we record detailed game context (quarter, remaining time in seconds, score margin), shot geometry (court coordinates in centimeters, derived shot distance from the basket, and simple spatial indicators such as side of the court), player profile information (anthropometric measurements including height, weight, wingspan, standing reach, hand length, and hand width, as well as physical test metrics such as lane agility time, three-quarter-court sprint time, and standing and maximum vertical leap, plus the nominal playing position PG, SG, SF, PF, or C), and schedule and travel variables (travel distance to the current game, number of rest days since the previous game, time-zone shifts between games, and indicators for dense schedule situations such as back-to-back games).

All data was obtained using the NBA Stats website, by either scraping it ourselves or relying on already scraped data from it. Using Jupyter Notebooks in SAS® Viya® Workbench, we derived two main versions of a master dataset combining these four feature blocks: a complete-case version, master_data_dropped, in which all shots with missing values in any modeling predictor were removed, yielding approximately 2.8 million shot attempts and including one-hot encoded position indicators; and an imputed version,

master_data_imputed, in which missing numeric predictors were filled using multivariate iterative imputation and explicit position dummies were omitted so that player roles are instead captured indirectly through continuous profile variables, resulting in roughly 4.5 million shots. These two datasets are structurally aligned but differ in their handling of missing data, allowing a direct comparison between a conservative complete-case strategy and an imputation-based approach that preserves nearly all observations.

DATA CLEANING AND VALIDATION

All data cleaning and validation were performed in SAS® Viya® Workbench before exporting the modeling datasets. We first addressed missing and invalid values. For the complete-case dataset, master_data_dropped, any observation with a missing predictor used in modeling was dropped, resulting in a clean dataset of about 2.8 million shots. For the imputed dataset, master_data_imputed, missing player profile predictors were filled using a multivariate normal approximation based on the joint distribution of the observed covariates, assuming anthropometric features follow roughly mean-normal patterns and physical measurements are reasonably approximated around their medians; categorical position dummies were not imputed and are omitted in this version.

In addition to handling missingness, we constructed several derived features and performed a series of quality checks: remaining time in the quarter was stored as a continuous variable (in seconds), while the quarter itself was kept as a categorical factor; score margin at the time of the shot was derived from the scoring history for each game; and travel distance, rest days, and time-zone shifts were retrieved from NBA Stats. To further ensure data integrity, we checked for duplicated records using combinations of game ID, player ID, and timestamps and removed any confirmed duplicates.

Basic summary statistics and simple visual inspections, such as histograms and heatmaps, were used to identify implausible values in both raw and derived features; when necessary, these values were filtered or capped. Taken together, these steps produced two clean and internally consistent datasets that share the same feature definitions but differ systematically in their approach to handling missing data, which becomes a recurring theme in the later model comparisons.

PROBLEM

The predictive task is to estimate whether a given shot attempt will be successful. We define a binary response variable MADE_SHOT that takes the value 1 if the shot is made and 0 otherwise. Each observation corresponds to one shot attempt and includes its associated context, player, and schedule information. Let X denote the feature vector for a shot (including distance, coordinates, quarter, remaining time, score margin, anthropometric and physical test measures, and schedule metrics), and let $Y \in \{0,1\}$ denote MADE_SHOT. Our goal is to learn a function $f(X)$ that approximates $P(Y = 1 \mid X)$, i.e., the probability that the shot goes in given its observed characteristics. This is a standard supervised binary classification problem. We focus on two broad questions: first, how well we can predict shot success using only contextual, player, and schedule features, and second, which predictors consistently emerge as important across different model classes. Because both classes (made and missed shots) are of interest and misclassifications in either direction matter, we emphasize the F1 score as the main performance metric, alongside overall accuracy and confusion matrices, to compare how different models and data-handling strategies trade off precision and recall.

ANALYSIS

All data preparation and feature engineering took place in SAS® Viya® Workbench. For model

fitting, we exported the prepared tables and built the predictive models in Python using scikit-learn, effectively giving us a SAS-to-Python pipeline. The preprocessing in the Python pipelines mirrors the feature definitions and transformations developed in SAS, ensuring that any differences in performance across models or datasets are not simply artifacts of inconsistent preprocessing. This setup lets us directly compare modeling strategies (logistic regression, decision trees, and SVMs) as well as missing-data strategies (complete-case vs. imputed) under a unified workflow, rather than mixing changes in code, features, and algorithms all at once.

FEATURE SET AND PREPROCESSING

To avoid information leakage and prevent the models from relying on identifiers without real predictive content, we removed all purely identifying variables (GAME_ID, PLAYER_ID, PLAYER_NAME, TEAM_ID, TEAM_NAME). We also excluded latitude and longitude coordinates related to travel (LAT, LON, D_LAT, D_LON), the estimated flight time (FLIGHT_TIME_MIN), and explicit home and away labels (HOME_TEAM, AWAY_TEAM) to reduce feature space dimensions and to keep the focus on interpretable basketball and schedule signals rather than proxy identifiers. Finally, we dropped the three-point indicator (IS_3PT) so that the models have to infer the effect of three-point attempts from shot location and distance instead of relying on a dedicated flag—this makes the comparison between linear and non-linear models more meaningful, because they must all "work" to extract that information from geometry.

The remaining predictors include shot geometry, quarter and remaining time, score margin, anthropometric and physical test measurements, schedule and travel metrics, and (in the complete-case dataset) position dummies. The quarter of the game is encoded using one-hot indicators. All continuous variables are standardized using z-score scaling, implemented with a ColumnTransformer that is combined with each classifier in a unified pipeline, so that cross-validation and final model fitting always apply exactly the same preprocessing steps, making performance metrics across models more directly comparable.

MISSING-DATA STRATEGIES IN MODELING

We fit each model class (regularized logistic regression, decision trees, and, on subsamples, SVMs) on both master_data_dropped and master_data_imputed, which allows a clear comparison of how sensitive each model type is to missing-data handling. On master_data_dropped, we gain explicit position information but lose observations with missing values, leading to a smaller but fully observed feature set. On master_data_imputed, we retain nearly all shots but rely on imputation and continuous player features to encode roles instead of hard-coded positional dummies.

Comparing results across these two versions helps gauge whether the extra sample size in the imputed data compensates for the uncertainty introduced by imputation, and whether models change their feature reliance when position is explicit versus embedded in continuous measurements. As we will see, performance differences between the two datasets are surprisingly small, suggesting that the imputation strategy is reasonably well-behaved and that the choice between dropped and imputed data is more about interpretability than about raw predictive accuracy.

REGULARIZED LOGISTIC REGRESSION

As a linear baseline, we used logistic regression with regularization via scikit-learn's SGDClassifier (loss = "log_loss"), which scales well to our multi-million-record datasets. We considered L1 (LASSO), which induces sparsity by shrinking some coefficients to zero, and L2 (Ridge), which shrinks coefficients without discarding features. The regularization parameter α was tuned over $\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$ using two-fold cross-validation with F1 as the objective, in pipelines that combined preprocessing and classifier fitting; for the best α, we refit on the full data and obtained cross-validated predictions via cross_val_predict, from

which we derived classification reports and confusion matrices. Across all four combinations (LASSO vs. Ridge on dropped vs. imputed data), results are very stable: accuracy is about 0.60 and macro F1 about 0.59 on both datasets, with the F1 for made shots around 0.54–0.55 and consistently higher recall for missed than for made shots, reflecting the intrinsic randomness of outcomes. L1 vs. L2 has only a modest impact: Ridge yields slightly better weighted F1, while LASSO produces sparser, more interpretable coefficient vectors. In every variant, shot distance is the dominant predictor with a large negative coefficient, game-context variables (quarter, remaining time) show small but consistent effects with earlier shots being more efficient, player size and position (when included) provide intuitive local adjustments near the basket, and a positive season coefficient suggests gradual league-wide improvements in shooting efficiency over time.

DECISION TREE CLASSIFIERS

We trained Decision Tree classifiers with RandomizedSearchCV (30 configurations, 3-fold cross-validation) on both complete-case and imputed datasets. The same hyperparameters were optimal for both: a moderately deep, strongly regularized tree (max_depth = 12, min_samples_split = 500, min_samples_leaf = 100, max_leaf_nodes = 2048). Cross-validated f1_weighted scores were nearly identical (0.605 vs. 0.602), indicating that imputation has little impact on tree performance, and test accuracy and weighted F1 were similarly close (~0.62 accuracy, ~0.60 weighted F1). The trees show very high recall for missed shots and low recall for made shots, generating many false negatives for made attempts and replicating the asymmetry seen in the logistic models; this pattern reflects both class imbalance and inherent noise in shot outcomes. While the resulting structures are interpretable—driven primarily by distance and game-time features—their predictive performance remains comparable to, rather than better than, logistic regression, and improving recall for made shots would likely require class rebalancing, cost-sensitive learning, or ensemble methods. ROC curves for the dropped and imputed versions are almost indistinguishable (AUC 0.6306 vs. 0.6276), both clearly above but not far from the random baseline, reinforcing the view that model class (linear vs. simple non-linear) matters less than the underlying signal in the features.

SUPPORT VECTOR MACHINES ON SUBSAMPLES

Support vector machines (SVMs) with non-linear kernels can learn more complex decision boundaries but are computationally expensive at full scale, so we trained them on 1% random subsamples of both master_data_dropped and master_data_imputed. Using the same preprocessing pipeline, we fit SVMs with RBF and polynomial kernels and tuned $C \in \{0.1, 1, 10\}$ and gamma $\in$ {"scale", "auto"} via two-fold cross-validated grid search with F1 as the objective. On these subsamples, the best SVMs achieve accuracy and macro F1 scores very similar to those of the logistic and tree models; allowing more flexible decision boundaries does not yield a clear performance jump. In a few cases, polynomial SVMs slightly improve recall for missed shots at the cost of worse performance on made shots, but these gains are small, inconsistent, and not robust across datasets. Given their high computational cost and lack of clear superiority—on either dropped or imputed data—we treat SVMs as a robustness check rather than a primary modeling tool, and their results suggest that most of the exploitable structure in this feature set is already captured by regularized logistic regression and decision trees.

VISUALIZATION (APPENDIX)

To analyze spatial shooting patterns, we built a custom Python shot chart function that manually draws all NBA half-court elements using the matplotlib package, as seen in the Visualization appendix. This ensures consistent geometry and styling across all visualizations and provides a reliable template for comparing players, positions, and seasons.

Using Stephen Curry as an example, the charts show a distinctly perimeter-heavy profile centered around the three-point arc, with made and missed shots clustering in the same areas, which is evidence of deliberate and stable shot selection. Scatter, hexbin, and heat map visualizations highlight both shot density and efficiency, while positional comparisons reveal clear differences between guards, wings, and centers. Curry's tendencies align with a high-usage perimeter guard archetype, particularly when contrasted with more interior-focused centers whose high-efficiency zones are closer to the rim.

Season-level trends show expected dips during the 2012 lockout-shortened season and rising volume and efficiency as his role expanded over time, mirroring league-wide shifts toward more three-point-heavy offenses.

SUGGESTIONS FOR FUTURE STUDIES

Future work could incorporate richer features such as player-tracking data (e.g., defender distance, movement, and shot type) and explore stronger but still interpretable models like tree ensembles or hierarchical/mixed-effects approaches. In parallel, more emphasis on probability calibration and on evaluating downstream impact (e.g., shot selection or lineup decisions) would shift the focus from pure accuracy to practical usefulness.

CONCLUSION

This project presents an end-to-end pipeline for predicting NBA shot success using large-scale shot-level data, from data preparation and missing-data handling in SAS® Viya® Workbench to model fitting in scikit-learn on two aligned datasets: a 2.8 million shot complete-case table and a 4.5 million shot imputed table. Across regularized logistic regression, decision trees, and SVMs on subsamples, the conclusions are remarkably consistent and robust to the missing-data strategy: shot distance is by far the dominant predictor, game context (quarter and remaining time) provides smaller but meaningful adjustments, player size and role have intuitive effects, and travel-related variables play at most a secondary role.

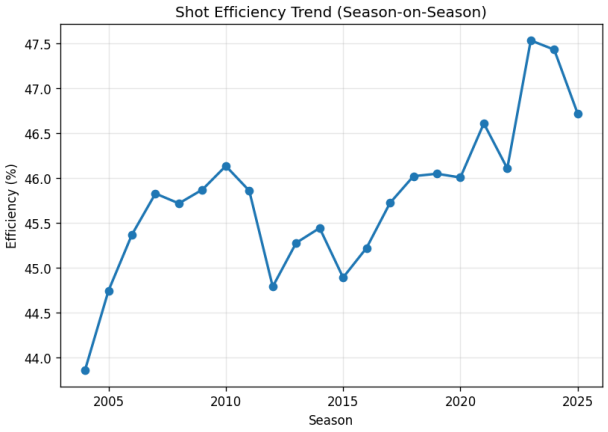
Predictive performance is moderate and similar across model classes, and more flexible SVMs do not deliver clear gains over simpler models, underscoring the inherent randomness of shot outcomes. At the same time, the models reliably recover stable, interpretable patterns in shooting efficiency, and the combined SAS–Python workflow offers a solid, extensible baseline for more advanced sports analytics work in the future.

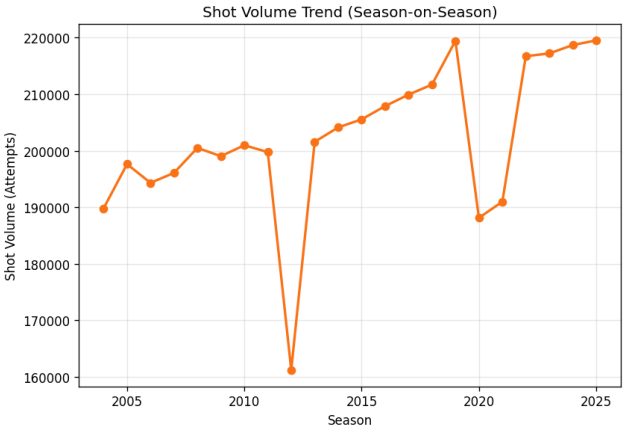
REFERENCES

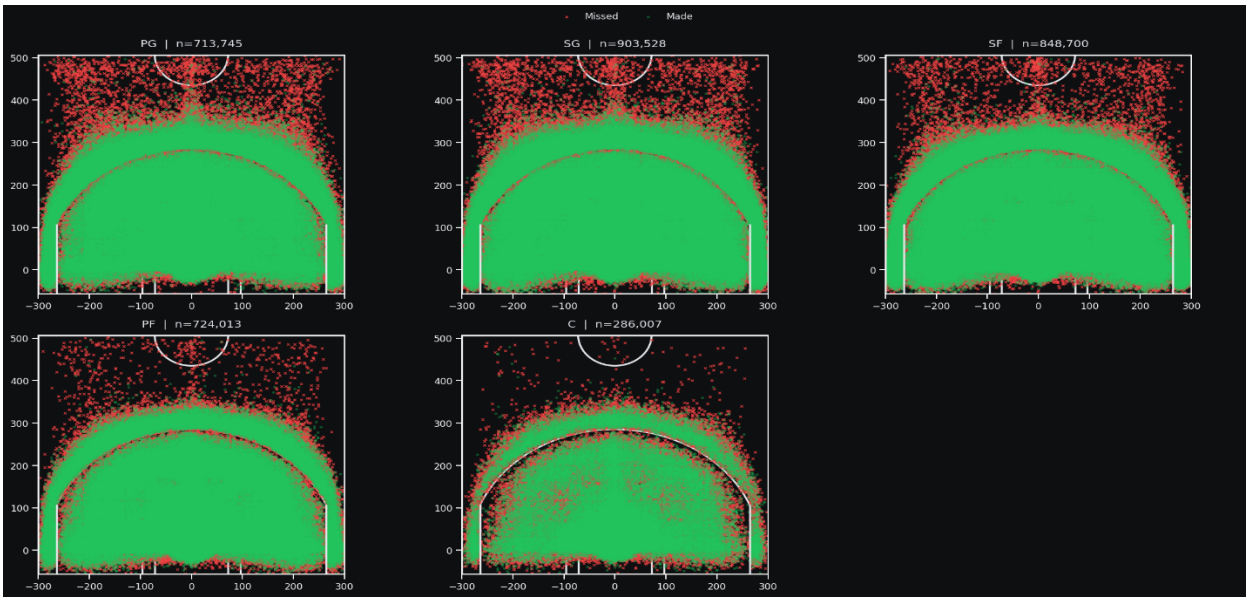
Website National Basketball Association (NBA). "NBA Stats." 22.02.2025. Available at https://www.nba.com/stats.

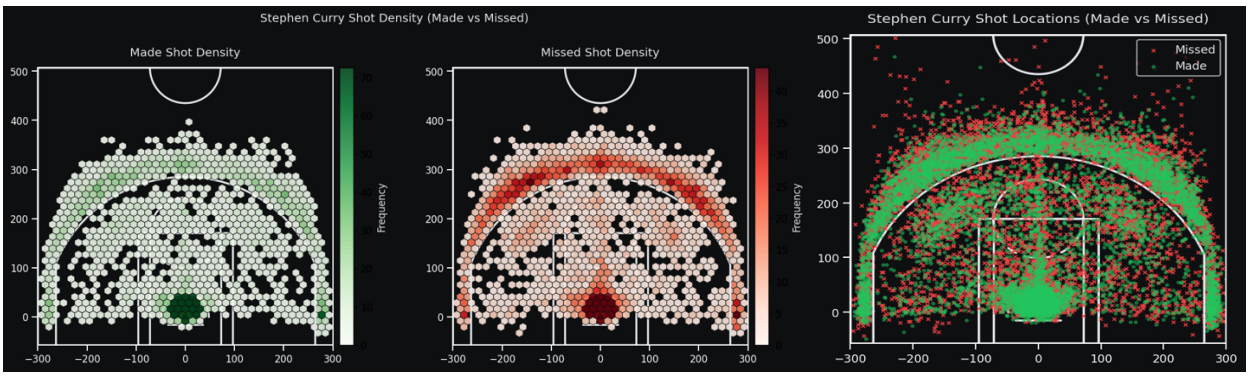
Website National Basketball Association (NBA). "NBA Court Dimensions" 22.02.2025. Available at https://official.nba.com/rule-no-1-court-dimensions-equipment/.

APPENDIX


Shot Efficiency Trend (Season-on-Season)
Efficiency (%)
Season


Shot Volume Trend (Season-on-Season)
Shot Volume (Attempts)
Season


Missed
Made
PG | n=713,745
SG | n=903,528
SF | n=848,700
PF | n=724,013
C | n=286,007


Stephen Curry Shot Density (Made vs Missed)
Made Shot Density
Missed Shot Density
Frequency
Stephen Curry Shot Locations (Made vs Missed)
Missed
Made